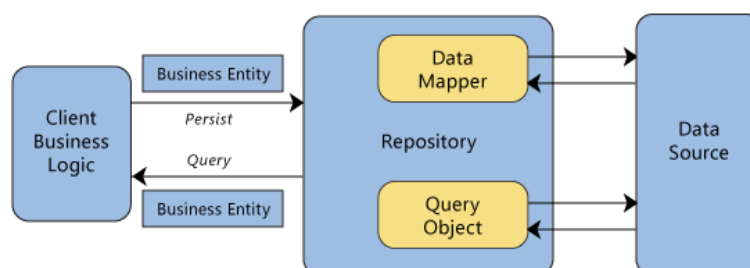


ORM, Repository

1. **Repository:** паттерн проектирования, применяемый при работе с данными; предназначен для разделения бизнес-логики и от слоя доступа к данным. Два подхода DDD: Database Drive Design (разработка начинается с разработки БД), Domain Drive Design (разработка начинается с проектирования бизнес-логики домена), Эрик Эванс). Repository ассоциируется с Domain Drive Design.
2. **Repository:** Data Mapper, Query Object.



3. Entity

```
public class WsRef
{
    [Key]
    public int Id { get; set; }
    public string? Url { get; set; }
    public string? Description { get; set; }
    public int? Plus { get; set; }
    public int? Minus { get; set; }
    public List<Comment>? Comments { get; set; } // для FK
}
```

4. **ORM:** Object-Relation Mapping, паттерн проектирования, применяется для работы с данными, концепция БД/ООП, ООП-интерфейс с БД, стандарт IT-индустрии.



5. **ORM/Java:** Hibernate, JPA
6. **ORM/Python/Django:** Django ORM, SQLAlchemy
7. **ORM/JavaScript/Node.js:** Sequelize, TypeORM
8. **ORM/Ruby:** ActiveRecord (on Rails), Sequel
9. **ORM/PHP:** Doctrine
10. **ORM/C#:** Entity Framework
11. **ORM:** БД, таблица, строка таблицы, структура таблицы, первичные ключи, внешние ключи.
12. Системные свойства реализаций:

Eager Loading — жадная загрузка связанных объектов сущности одновременно с основным объектом; предотвращает множество дальнейших запросов, плохо сказывается на скорости выполнения чтения;

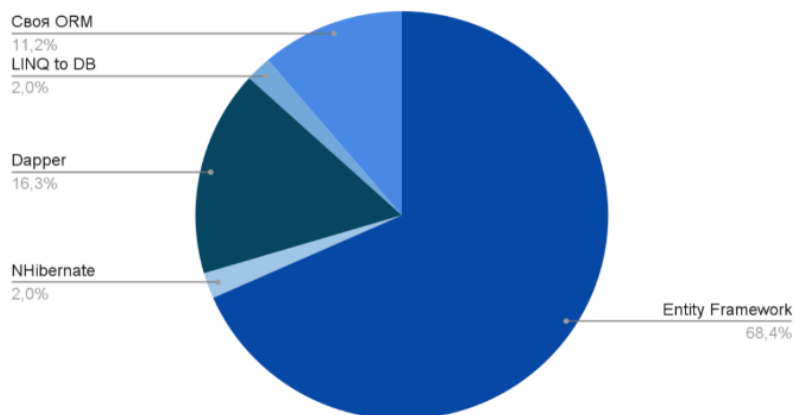
Lazy Loading — ленивая загрузка, позволяющая оптимизировать процесс чтения и получать связанные объекты только при необходимости;

Identity Map — жадное получение объекта из базы данных 1 раз в рамках сессии; позволяет избежать возможного противоречия в данных, возникающих из-за нескольких запросов;

Unit of Work — собирает все изменения во всех объектах и позволяет выполнить их в рамках одной транзакции; благоприятно сказывается на консистентности данных и количестве запросов.

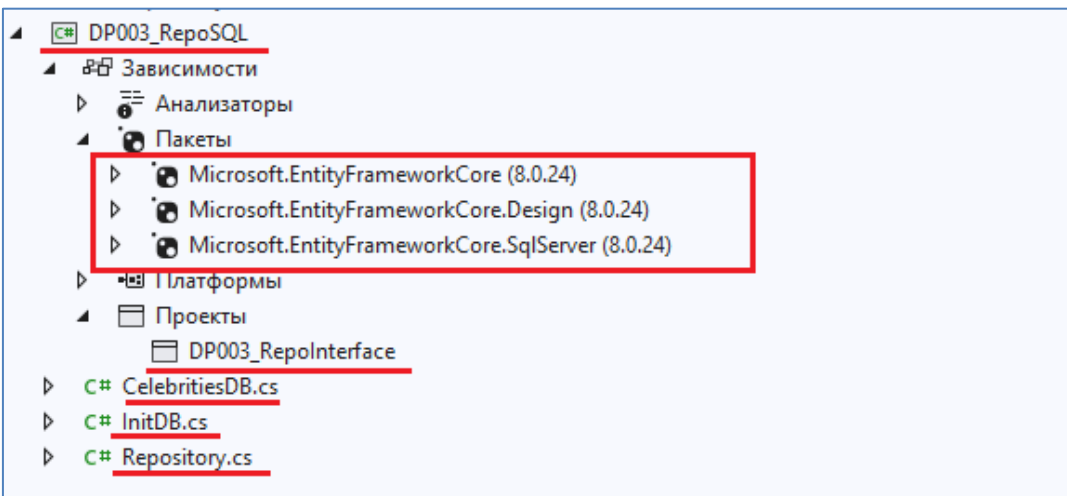
13. **ORM:** применение

Какие ORM предпочитают



14. ORM: <https://metanit.com/sharp/adonetcore/2.4.php>
15. <https://habr.com/ru/articles/667078/>
16. Data Mapper
17. **ADO.NET**: <https://metanit.com/sharp/adonet/1.1.php>
18. **Entity-Component-System(ECS) pattern**:
<https://habr.com/ru/articles/665276/>
19. Пример

```
namespace DP003_RepoInterface
{
    public interface ICelebrity<T> : IDisposable
    {
        List<T> GetAllCelebrities();           // получить все Знаменитости
        T? GetCelebrityById(int Id);           // получить Знаменитость по Id
        bool DelCelebrity(int id);             // удалить Знаменитость по Id
        bool AddCelebrity(T celebrity);        // добавить Знаменитость
        int AddCelebrityAndGetId(T celebrity); // добавить Знаменитость и получить его Id
        bool UpdCelebrity(int id, T celebrity); // изменить Знаменитость по Id
        int GetCelebrityIdByName(string name); // получить первый Id по вхождению подстроки
        int SaveChanges();                    // сохранить изменения = количество изменений
    }
}
```



```
<Project Sdk="Microsoft.NET.Sdk">

  <PropertyGroup>
    <TargetFramework>net8.0</TargetFramework>
    <ImplicitUsings>enable</ImplicitUsings>
    <Nullable>enable</Nullable>
  </PropertyGroup>

  <ItemGroup>
    <PackageReference Include="Microsoft.EntityFrameworkCore" Version="8.0.24" />
    <PackageReference Include="Microsoft.EntityFrameworkCore.Design" Version="8.0.24">
      <PrivateAssets>all</PrivateAssets>
      <IncludeAssets>runtime; build; native; contentfiles; analyzers; buildtransitive</IncludeAssets>
    </PackageReference>
    <PackageReference Include="Microsoft.EntityFrameworkCore.SqlServer" Version="8.0.24" />
  </ItemGroup>

  <ItemGroup>
    <ProjectReference Include="..\DP003_RepoInterface\DP003_RepoInterface.csproj" />
  </ItemGroup>

</Project>
```

```

public class Celebrity // Entity
{
    public Celebrity(int? Id, string Firstname, string Surname, string PhotoPath)
    {
        this.Id = Id;
        this.Firstname = Firstname;
        this.Surname = Surname;
        this.PhotoPath = PhotoPath;
    }

    public int? Id; // идентификатор
    public string Firstname = null!; // имя
    public string Surname = null!; // фамилия
    public string PhotoPath = null!; // путь к файлу фотографии
}

```

```

public class CelebritiesDB:DbContext // БД
{
    string connectionString = "Server=██████████;Database=DPCelebrities;User ID=██████████;Password=██████████;TrustServerCertificate=true";
    public CelebritiesDB(string connectionString)
    {
        this.connectionString = connectionString;
    }
    public DbSet<Celebrity> Celebrities => Set<Celebrity>();

    protected override void OnConfiguring(DbContextOptionsBuilder optionsBuilder)
    {
        if (!optionsBuilder.IsConfigured)
        {
            optionsBuilder.UseSqlServer(connectionString);
        }
    }

    protected override void OnModelCreating(ModelBuilder modelBuilder)
    {
        modelBuilder.Entity<Celebrity>(entity =>
        {
            entity.HasKey(c => c.Id);
            entity.Property(c => c.Firstname).IsRequired().HasMaxLength(100);
            entity.Property(c => c.Surname).IsRequired().HasMaxLength(100);
            entity.Property(c => c.PhotoPath).HasMaxLength(200);
        });
    }
}

```

```

namespace DP003_RepoSQL
{
    public class Repository : ICelebrity<Celebrity>
    {
        CelebritiesDB celebrities;

        Repository(string connectionString) => this.celebrities = new CelebritiesDB(connectionString);

        public static ICelebrity<Celebrity> Create(string basepath) { return new Repository(basepath); }

        public bool AddCelebrity(Celebrity celebrity)
        {
            celebrity.Id = null;
            this.celebrities.Celebrities.Add(celebrity);
            return this.SaveChanges() > 0;
        }

        public int AddCelebrityAndGetId(Celebrity celebrity)
        {
            this.AddCelebrity(celebrity);
            this.celebrities.SaveChanges();
            return celebrity.Id ?? -1;
        }
    }
}

```

```

        public bool DelCelebrity(int id)
        {
            Celebrity? c = this.celebrities.Celebrities.FirstOrDefault<Celebrity>(c => c.Id == id);
            if (c != null) this.celebrities.Celebrities.Remove(c);
            return this.SaveChanges() > 0;
        }

        public void Dispose() => this.celebrities.Dispose();

        public List<Celebrity> GetAllCelebrities() => this.celebrities.Celebrities.ToList<Celebrity>();

        public Celebrity? GetCelebrityById(int Id) => this.celebrities.Celebrities.FirstOrDefault<Celebrity>(c => c.Id == Id);

        public int GetCelebrityIdByName(string name)
        {
            Celebrity? c = this.celebrities.Celebrities.FirstOrDefault<Celebrity>(c=>c.Surname.Equals(name));
            return c != null? c.Id??-1 : -1;
        }

        public int SaveChanges() => this.celebrities.SaveChanges();
    }
}

```

```

        public bool UpdCelebrity(int id, Celebrity celebrity)
        {
            bool rc = false;
            Celebrity? c = GetCelebrityById(id);
            if (c != null)
            {
                c.Firstname = celebrity.Firstname;
                c.Surname = celebrity.Surname;
                c.PhotoPath = celebrity.PhotoPath;
                rc = this.celebrities.SaveChanges() > 0;
            }
            return rc;
        }
    }
}

```

```

using DP003.RepoInterface;
//using Repo = DP003.RepoJSON;
using Repo = DP003.RepoSQL;
internal class Program
{
    private static async Task Main(string[] args)
    {
        await (new Repo.InitDB("Server=██████████;Database=DPCelebrities;User ID=██████████;Password=██████████;TrustServerCertificate=true")).InitializeAsync();
        using (ICelebrityRepo celebrityRepo = Repo.Repository.Create("Server=██████████;Database=DPCelebrities;User ID=██████████;Password=██████████;TrustServerCertificate=true"))
        {
            Console.WriteLine("----- GetAllCelebrities -----");
            {
                List<Repo.Celebrity> celebrities = celebrityRepo.GetAllCelebrities();
                celebrities.ForEach(c => Console.WriteLine($"{c.Id}, {c.Firstname}, {c.Surname}, {c.PhotoPath}"));
            }
            Console.WriteLine("----- AddCelebrity -----");
            {
                celebrityRepo.AddCelebrity(new Repo.Celebrity(0, "Marlin", "Minsky", "/Photo/Minsky.jpg"));
                List<Repo.Celebrity> celebrities = celebrityRepo.GetAllCelebrities();
                celebrities.ForEach(c => Console.WriteLine($"{c.Id}, {c.Firstname}, {c.Surname}, {c.PhotoPath}"));
            }
            Console.WriteLine("----- GetCelebrityByIdByName, UpdCelebrity -----");
            {
                int id = celebrityRepo.GetCelebrityByIdByName("Marlin");
                if (id >= 0) celebrityRepo.UpdCelebrity(id, new Repo.Celebrity(0, "Marvin", "Minsky", "/Photo/Minsky.jpg"));
                List<Repo.Celebrity> celebrities = celebrityRepo.GetAllCelebrities();
                celebrities.ForEach(c => Console.WriteLine($"{c.Id}, {c.Firstname}, {c.Surname}, {c.PhotoPath}"));
            }
            Console.WriteLine("----- GetCelebrityByIdByName, DelCelebrity -----");
            {
                int id = celebrityRepo.GetCelebrityByIdByName("Minsky");
                if (id >= 0) celebrityRepo.DelCelebrity(id);
                List<Repo.Celebrity> celebrities = celebrityRepo.GetAllCelebrities();
                celebrities.ForEach(c => Console.WriteLine($"{c.Id}, {c.Firstname}, {c.Surname}, {c.PhotoPath}"));
            }
            Console.WriteLine("----- AddCelebrityAndGetId -----");
            {
                int id = celebrityRepo.AddCelebrityAndGetId(new Repo.Celebrity(0, "Marvin", "Minsky", "/Photo/Minsky.jpg"));
                Console.WriteLine($"Minsky Id = {id}");
                List<Repo.Celebrity> celebrities = celebrityRepo.GetAllCelebrities();
                celebrities.ForEach(c => Console.WriteLine($"{c.Id}, {c.Firstname}, {c.Surname}, {c.PhotoPath}"));
            }
        }
    }
}

```

```

public class InitDB
{
    CelebritiesDB celebritiesDB;
    public InitDB(string connectionString) => this.celebritiesDB = new CelebritiesDB(connectionString);
    public async Task InitializeAsync()
    {
        await celebritiesDB.Database.EnsureCreatedAsync(); // Создаём БД и схему, если их ещё нет
        if (!celebritiesDB.Celebrities.Any()) // Если нет данных - добавляем
        {
            var celebrities = new[]
            {
                new Celebrity(0, "Noam", "Chomsky", "/Photo/Chomsky.jpg"),
                new Celebrity(0, "Tim", "Berners-Lee", "/Photo/Berners-Lee.jpg"),
            };
            await celebritiesDB.Celebrities.AddRangeAsync(celebrities);
        }
        await celebritiesDB.SaveChangesAsync();
    }
}

```

20. КОНЕЦ